DESIGN PROJECT – I · SEMESTER III

CryptoGuard

A Measured Password and File-Security Workbench

A project report submitted in partial fulfilment of the requirements of Design Project – I (Semester III) for the degree of

BACHELOR OF TECHNOLOGY

with specialisation in Cyber Security

Submitted by

Malepati Sree Harsha

Register No. 23CU0340018

DISC

HINDUSTAN INSTITUTE OF TECHNOLOGY AND SCIENCE

(Deemed to be University)

Padur, Kelambakkam, Chennai – 603 103

Academic Year 2024–25

DECLARATION

I, **Malepati Sree Harsha** (Register No. 23CU0340018), hereby declare that the project report entitled **“CryptoGuard: A Measured Password and File-Security Workbench”**, submitted as Design Project – I in Semester III of the Bachelor of Technology programme with specialisation in Cyber Security, DISC, Hindustan Institute of Technology and Science, during the academic year 2024–25, is a record of my own work.

Where the work of others has been used, it is acknowledged and cited in the references. All experiments were performed on systems I own, using bundled or synthetic data, and no third-party system was tested without authorization.

Place: Chennai

Date: _____

Malepati Sree Harsha

Register No. 23CU0340018

ACKNOWLEDGEMENT

I thank the faculty of DISC, Hindustan Institute of Technology and Science, for the course structure, laboratory facilities and feedback that shaped this design project, and the institution for the opportunity to work on an applied security problem.

I also acknowledge the open-source communities behind Python, Flask, scikit-learn, the cryptography library, OpenSSL, John the Ripper, Wireshark/TShark, Suricata, Nmap and OWASP ZAP, whose tools and documentation made this work possible. Finally, I thank my family and friends for their support.

Malepati Sree Harsha

ABSTRACT

Password-strength meters usually convert a password into a crack-time estimate using a guess rate that the tool simply assumes. In practice the rate an attacker achieves depends far more on how the *server* stores the password than on a few extra characters typed by the user. This project, CryptoGuard, is a local, privacy-preserving web workbench that makes these trade-offs measurable. It combines (i) an explainable password-strength analyser, (ii) cryptographically secure password and passphrase generation, (iii) authenticated file encryption with AES-256-GCM and PBKDF2-HMAC-SHA256 key derivation, (iv) file-integrity digests and hash-format identification, (v) a bounded John the Ripper demonstration on bundled sample hashes, and (vi) a novel **measured key-derivation benchmark** that times MD5, SHA-1, SHA-256, PBKDF2 (200,000 and 600,000 iterations) and scrypt on the host and expresses each scheme's cost as an equivalent number of extra password characters.

The system is implemented in Python (Flask) with a responsive single-page interface, packaged as a non-root Docker container, and hardened with a strict Content-Security-Policy, OWASP security headers, request-ID logging that never records request bodies, JSON error handling and per-client rate limits. Evaluation on the Docker image shows that one CPU core computes about 3,516,182 MD5 guesses per second but only 15.2 PBKDF2-600k guesses per second, a 231,328-fold slowdown worth about 2.71 extra random characters from a 95-symbol alphabet. John the Ripper recovered 8 of 9 bundled weak hashes from a 16-word list. All 14 automated tests pass, and non-KDF endpoints answer with a median latency under 5 ms at more than 1,500 requests per second. The work shows that storage-side choices deserve the same visibility as user-side password rules, and it provides a reproducible way to demonstrate this in a classroom or interview.

Keywords: password security, key derivation function, PBKDF2, scrypt, AES-GCM, authenticated encryption, John the Ripper, secure web application, Flask, Docker

CONTENTS

Declaration ii

Acknowledgement iii

Abstract iv

List of Figures vii

List of Tables viii

Chapter 1 Introduction 1

1.1 Background 1

1.2 Motivation 1

1.3 Problem Statement 1

1.4 Objectives 2

1.5 Scope and Ethics 2

1.6 Organisation of the Report 2

Chapter 2 Literature Survey 3

2.1 Password Strength Estimation 3

2.2 Password Storage and Key Derivation 3

2.3 Authenticated File Encryption 3

2.4 Existing Tools 3

2.5 Research Gap 4

Chapter 3 System Requirements Specification 5

3.1 Functional Requirements 5

3.2 Non-Functional Requirements 5

3.3 Hardware and Software Requirements 6

3.4 Feasibility 6

Chapter 4 System Design 7

4.1 Architecture 7

4.2 Data Flow Diagrams 7

4.3 UML Diagrams 8

4.4 Deployment Design 10

4.5 Encrypted Envelope Format 11

Chapter 5 Methodology and Implementation 12

5.1 Technology Stack and Justification 12

5.2 Stage 1 — Project Skeleton and API 12

5.3 Stage 2 — Explainable Password Analysis 12

5.4 Stage 3 — Secure Secret Generation 13

5.5 Stage 4 — Authenticated Encryption 13

5.6 Stage 5 — Integrity and Hash Identification 14

5.7 Stage 6 — Bounded John the Ripper Demonstration....14
5.8 Stage 7 — Measured KDF Benchmark (Novel Component)....15
5.9 Stage 8 — HTTP Hardening, Logging and Rate Limiting....16
5.10 Stage 9 — User Interface....16
5.11 Stage 10 — Packaging and Continuous Integration....16
5.12 Secure Coding Practices Applied....17
Chapter 6 Testing and Validation....18
6.1 Test Strategy....18
6.2 Automated Test Cases....18
6.3 Functional, Usability and Performance Checks....19
6.4 Security Testing Notes....19
Chapter 7 Results and Discussion....21
7.1 Working System....21
7.2 Measured KDF Cost....22
7.3 Strength Analyser Behaviour....24
7.4 Performance....25
7.5 Usability on Mobile....26
7.6 Limitations....26
Chapter 8 Conclusion and Future Scope....27
8.1 Conclusion....27
8.2 Future Scope....27
References....28
Appendix A Installation and User Manual....29
Appendix B Demonstration Script....30
Appendix C API Reference....31
Appendix D Repository Structure....32

LIST OF FIGURES

Figure 4.1 Layered architecture of CryptoGuard 7
Figure 4.2 DFD level 0 (context diagram) 8
Figure 4.3 DFD level 1 8
Figure 4.4 Use-case diagram 9
Figure 4.5 Sequence diagram for AES-256-GCM file encryption 9
Figure 4.6 Class and module diagram 10
Figure 4.7 Activity diagram of password-strength analysis 10
Figure 4.8 Deployment diagram 11
Figure 7.1 CryptoGuard landing view 21
Figure 7.2 Live password analysis with checks and crack-time scenarios 21
Figure 7.3 AES-256-GCM encryption producing a .cguard envelope 22
Figure 7.4 John the Ripper cracking 8 of 9 bundled sample hashes 22
Figure 7.5 Guesses per second per scheme (log scale) 23
Figure 7.6 Benchmark rendered in the user interface 24
Figure 7.7 Latency per endpoint (p50 and p95) 25
Figure 7.8 Layout at 390 px width 26

LIST OF TABLES

Table 2.1 Comparison of existing tools with CryptoGuard 3
Table 3.1 Functional requirements 5
Table 3.2 Non-functional requirements and how they are verified 5
Table 3.3 Hardware and software requirements 6
Table 4.1 Fields of the .cguard envelope 11
Table 5.1 Technology choices 12
Listing 5.2 Password strength analysis (cryptoguard/crypto_core.py) 12
Listing 5.3 Key derivation and AES-256-GCM (cryptoguard/crypto_core.py) 13
Listing 5.4 John the Ripper adapter, beginning (cryptoguard/tools.py) 14
Listing 5.5 Benchmark core (cryptoguard/kdf_bench.py) 15
Listing 5.6 Rate-limiting decorator (cryptoguard/web_security.py) 16
Table 6.1 Test levels used 18
Table 6.2 Automated test cases and results 18
Table 6.3 Functional and non-functional checks 19
Table 7.1 Measured guess rates (Linux aarch64, Python 3.12.16, one core) 23
Table 7.2 Load-test results against the Docker container (100 requests, 8 workers per endpoint) 25

CHAPTER 1

INTRODUCTION

1.1 Background

Passwords remain the dominant authentication mechanism on the web, and credential theft is a consistent leading cause of breaches. Two independent decisions determine how well a password resists an attacker who has stolen a password database: the user's choice of password, and the service's choice of how to store it. Users receive constant advice about the first (length, character classes, uniqueness); the second is invisible to them.

Cryptography offers well-understood building blocks for the second decision. Slow, salted key-derivation functions (KDFs) such as PBKDF2 [2], [4] and memory-hard functions such as scrypt [5] and Argon2 [11] deliberately make each guess expensive. Authenticated encryption modes such as AES-GCM [3] protect both the confidentiality and the integrity of files. These ideas are standard in industry guidance [1], [9] but are rarely made *tangible* to the people who must choose or audit them.

1.2 Motivation

Most strength meters display a single crack time such as “centuries” without stating the attacker model behind it. Research shows such estimates are often poorly aligned with real cracking behaviour [6], [7], [15], and that users' perceptions of password strength differ from reality [12]. The guess rate hidden inside the estimate can vary by five or more orders of magnitude depending on the storage scheme. A student or developer therefore cannot tell from a typical meter whether a password is weak, or whether the *system* storing it is weak.

A second motivation is privacy. Many online password checkers and file-encryption websites require the user to send secrets to a third-party server. An educational tool for security hygiene should itself demonstrate good hygiene: process data locally and never log it.

1.3 Problem Statement

To design and implement a local, secure web application that lets a user evaluate passwords, generate strong secrets, encrypt and verify files, and observe real password cracking, while quantifying with measurements on the host machine how much the server-side password storage scheme changes an attacker's guessing speed, expressed in a unit users already understand: password characters.

1.4 Objectives

1. Implement an explainable password-strength analyser that reports entropy, policy checks, detected weaknesses and crack-time scenarios with their assumptions stated.
2. Generate passwords and passphrases with an operating-system CSPRNG (Python `secrets`).
3. Provide authenticated encryption of files and text using AES-256-GCM with PBKDF2-HMAC-SHA256 key derivation and a self-describing, downloadable envelope.
4. Compute integrity digests (SHA-256, SHA-1, MD5) and identify hash formats using `hashid` and built-in rules.
5. Demonstrate dictionary attacks safely with John the Ripper restricted to bundled sample hashes.
6. Design and evaluate a measured KDF benchmark that converts each storage scheme's cost into equivalent password characters.
7. Apply secure-coding practice: input validation, security headers, safe logging, error handling and rate limiting, verified by automated tests.
8. Package the system reproducibly with Docker and continuous integration, and evaluate functionality, security and performance.

1.5 Scope and Ethics

CryptoGuard is an educational, single-user tool that runs on the user's own machine and binds only to the loopback interface. It is not a password manager or a production encryption service, and it has not been independently audited. The cracking demonstration accepts no user input: it only reads the sample hash file and word list shipped with the repository. No experiment in this report involved third-party systems or real user credentials.

1.6 Organisation of the Report

Chapter 2 surveys existing tools and literature and identifies the research gap. Chapter 3 specifies requirements. Chapter 4 presents the system design with architecture, data-flow and UML diagrams. Chapter 5 describes the implementation stage by stage. Chapter 6 covers testing and validation. Chapter 7 presents results and discussion, and Chapter 8 concludes with future scope. Appendices contain the installation guide, demonstration script, API reference and repository structure.

CHAPTER 2

LITERATURE SURVEY

2.1 Password Strength Estimation

Early strength measures counted character classes and multiplied length by log2 of the alphabet size, an approach popularised by older NIST guidance. Weir et al. [8] showed, by attacking large leaked password sets, that such entropy scores are a poor predictor of resistance to real attacks. Kelley et al. [15] measured strength by simulating cracking algorithms and found policy-based scores disagree with guess numbers. Bonneau [6] analysed 70 million passwords and proposed guessing-based metrics. Wheeler's zxcvbn [7] estimates guess numbers by matching dictionary words, keyboard patterns, dates and l33t substitutions, and is deployed widely. Ur et al. [12] found that users misjudge which passwords are strong.

Current NIST guidance (SP 800-63B [1]) moved away from composition rules toward length, blocklists of compromised passwords, and rate limiting, and requires verifiers to store passwords with a salted, iterated one-way KDF.

2.2 Password Storage and Key Derivation

PBKDF2 [2], [4] applies an HMAC many times so that each guess costs a configurable number of iterations. OWASP's Password Storage Cheat Sheet [9] recommends at least 600,000 iterations for PBKDF2-HMAC-SHA256, or a memory-hard function. scrypt [5] adds a large memory requirement to resist GPU and ASIC parallelism; Argon2 [11], winner of the Password Hashing Competition, generalises this design. Fast general-purpose hashes (MD5, SHA-1, SHA-256) are unsuitable for passwords because they are optimised for speed.

2.3 Authenticated File Encryption

Galois/Counter Mode [3] provides confidentiality and a 128-bit authentication tag. Decryption fails if the ciphertext, nonce or key is wrong, which prevents silent acceptance of tampered data. GCM requires a unique nonce per key; deriving a fresh key per file from a random salt, as in this project, keeps nonce reuse risk negligible.

2.4 Existing Tools

Table 2.1: Comparison of existing tools with CryptoGuard

Tool / approach	What it does well	Limitation addressed here
Browser / password-manager meters (e.g. zxcvbn [7])	Realistic pattern-aware estimates	Assume one attacker speed; storage scheme invisible

Tool / approach	What it does well	Limitation addressed here
Online password checkers	Convenient	Send secrets to a third party; no stated model
John the Ripper [10] / Hashcat	Real cracking power	Expert CLI tools; not connected to strength feedback
7-Zip, VeraCrypt (AES)	Strong, audited encryption	Separate tools; parameters hidden from learners
NIST / OWASP guidance [1], [9]	Authoritative parameters	Text only; no way to observe the effect
CryptoGuard (this work)	Local, explainable, integrated	**Measures KDF cost on the host in password-character units**

2.5 Research Gap

The surveyed work treats the attacker's guess rate either as a fixed assumption (meters) or as a property of attack hardware (cracking tools). No widely used educational tool lets a learner (a) measure, on their own hardware, how storage schemes change the guess rate, and (b) compare that change in the same unit as user-side advice. This project addresses that gap with a small, reproducible benchmark and an integrated, local workbench. The contribution is deliberately modest: it is an engineering and educational contribution, not a new cryptographic primitive or a new strength estimator.

CHAPTER 3

SYSTEM REQUIREMENTS SPECIFICATION

3.1 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement	Endpoint / module
FR-1	Analyse a password (≤ 256 chars) and return score, label, entropy, checks, issues and crack-time scenarios	POST /api/analyze
FR-2	Generate a password (12–64 chars, optional symbols) or passphrase (4–8 words) using a CSPRNG	GET /api/generate
FR-3	Encrypt text or a file (≤ 8 MB) with AES-256-GCM and return a .cguard envelope	POST /api/encrypt
FR-4	Decrypt an envelope with the passphrase; reject wrong keys or tampered data	POST /api/decrypt
FR-5	Compute SHA-256, SHA-1 and MD5 digests of text or a file	POST /api/integrity
FR-6	Identify the probable format of a pasted hash	POST /api/identify
FR-7	Run John the Ripper on bundled sample hashes only and report cracked passwords	GET /api/john
FR-8	Measure hash/KDF cost on the host and convert it to equivalent characters	POST /api/kdf-bench
FR-9	Report installed tool status and OpenSSL details	GET /api/tools, /api/openssl
FR-10	Export or print a report of the current analysis	Browser UI

3.2 Non-Functional Requirements

Table 3.2: Non-functional requirements and how they are verified

ID	Category	Requirement	Verification
NFR-1	Privacy	No user input is written to disk or logs	Code review; logs contain method/path/status only
NFR-2	Security	CSP, X-Frame-Options, nosniff, Referrer-Policy on every response	tests/test_security.py
NFR-3	Security	Bounded input sizes; rate limits on expensive endpoints	Validation tests; 429 test
NFR-4	Performance	Non-KDF endpoints p95 < 50 ms on a laptop	scripts/loadtest.py (Table 7.3)
NFR-5	Usability	Usable from 320 px to desktop widths; 44 px touch targets	Screenshots at 390 px
NFR-6	Portability	One-command start on any OS with Docker	docker compose up
NFR-7	Reliability	Container health check; graceful degradation if a tool is missing	HEALTHCHECK; tool-status API
NFR-8	Maintainability	Modular code, automated tests, CI on every push	GitHub Actions workflow

3.3 Hardware and Software Requirements

Table 3.3: Hardware and software requirements

Item	Minimum	Used in this work
Processor	Any 64-bit x86 or ARM CPU	Apple Silicon (arm64)
Memory	4 GB RAM	8 GB+
Operating system	Windows 10+, macOS 12+, or Linux	macOS; container runs Debian (python:3.12-slim)
Runtime	Docker Desktop 4.x, or Python 3.9+	Docker 29; Python 3.12 in container
Libraries	Flask ≥ 3.0, cryptography ≥ 42, hashID	As pinned in requirements.txt
Security tools	John the Ripper, OpenSSL (in image)	John (Debian package), OpenSSL 3.5.7 9 Jun 2026
Browser	Any modern browser	Chromium, Safari

3.4 Feasibility

- **Technical:** all components are mature open-source libraries; the cryptography is delegated to OpenSSL via the `cryptography` package rather than implemented by hand.
- **Economic:** zero licence cost; runs on a student laptop.
- **Operational:** a single `docker compose up` command starts the system; a macOS double-click launcher is included.
- **Legal and ethical:** no third-party systems are contacted; the cracking demonstration only uses bundled synthetic hashes.

CHAPTER 4

SYSTEM DESIGN

4.1 Architecture

CryptoGuard follows a layered architecture (Figure 4.1). The presentation layer is a single self-contained HTML page. The API layer is a Flask application whose cross-cutting security concerns live in `web_security.py`. The domain layer contains pure-Python modules with no web dependencies (so they can be unit-tested directly), and the tool layer wraps external programs behind bounded subprocess calls.

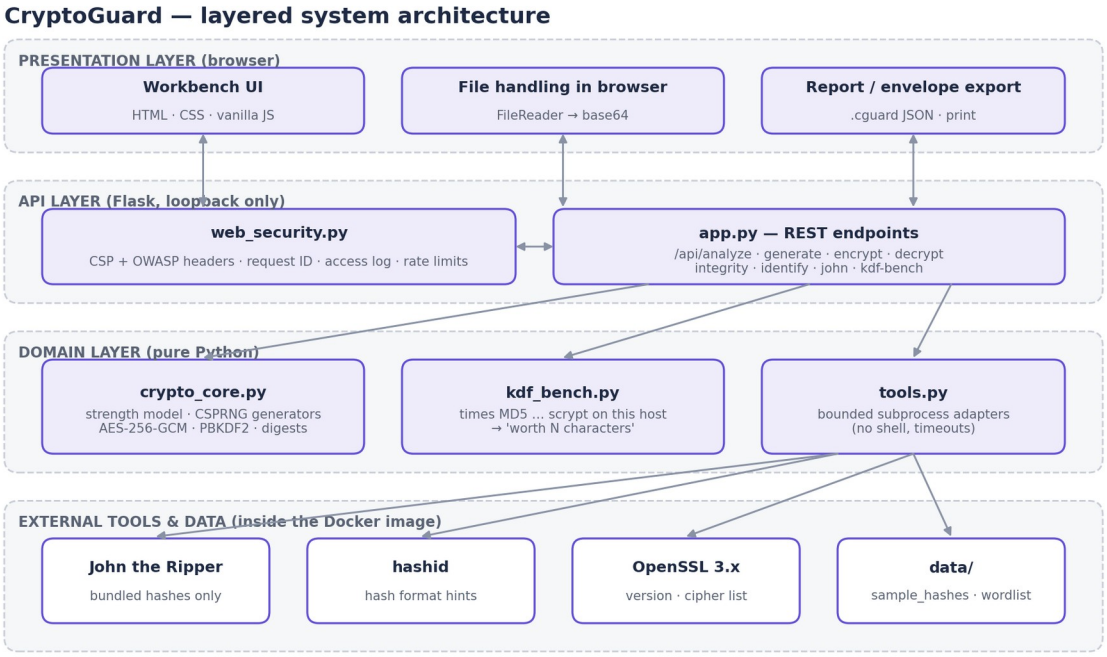


Figure 4.1: Layered architecture of CryptoGuard

4.2 Data Flow Diagrams

The context diagram (Figure 4.2) shows the system boundary: the only external entities are the user, the locally installed tools and the bundled sample data. The level-1 diagram (Figure 4.3) decomposes the system into six processes. No process writes user input to persistent storage.

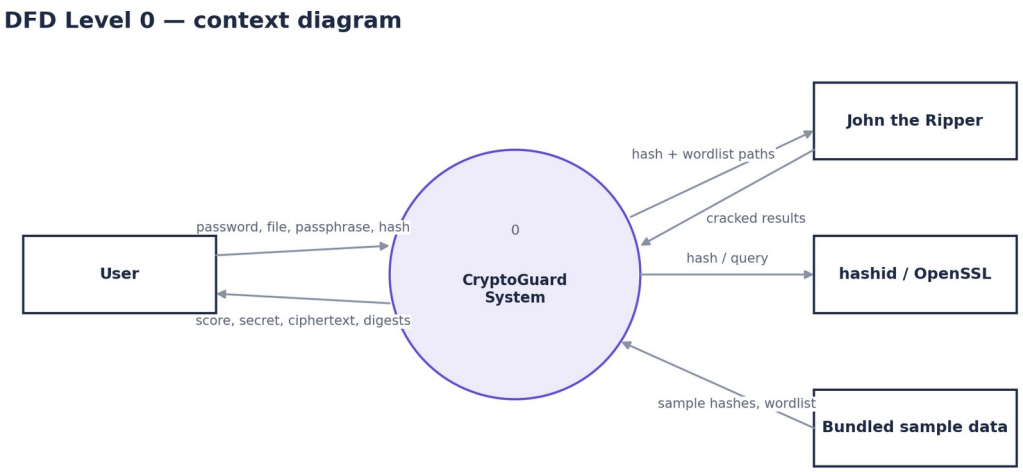


Figure 4.2: DFD level 0 (context diagram)

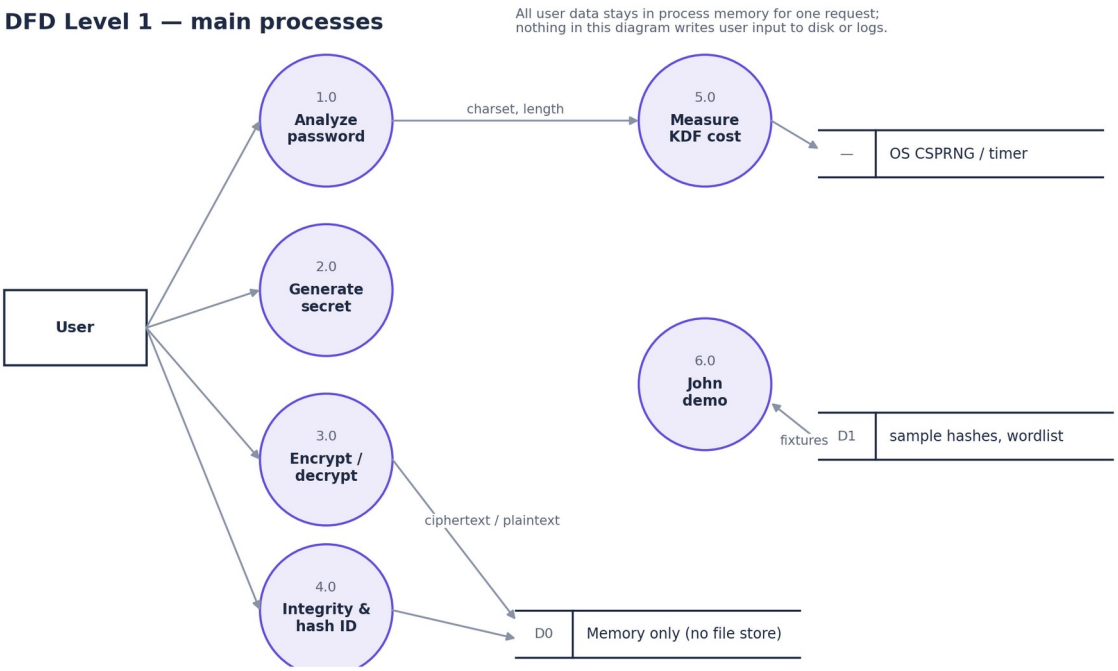


Figure 4.3: DFD level 1

4.3 UML Diagrams

Figure 4.4 shows the use cases for the two actors. Figure 4.5 traces the most security-sensitive interaction, encrypting a file, through every layer. Figure 4.6 shows the module structure, and Figure 4.7 the activity flow of the strength analyser.

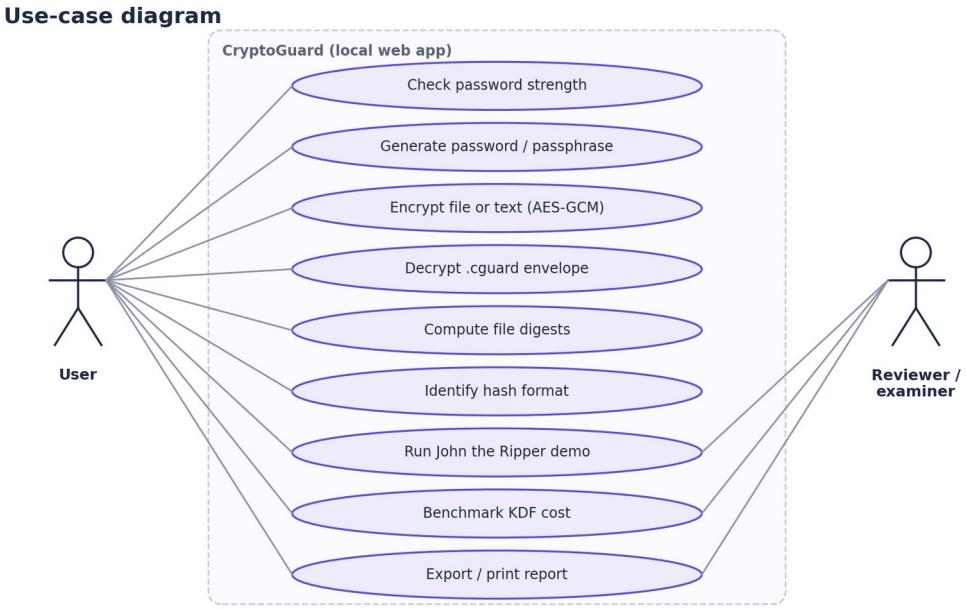


Figure 4.4: Use-case diagram

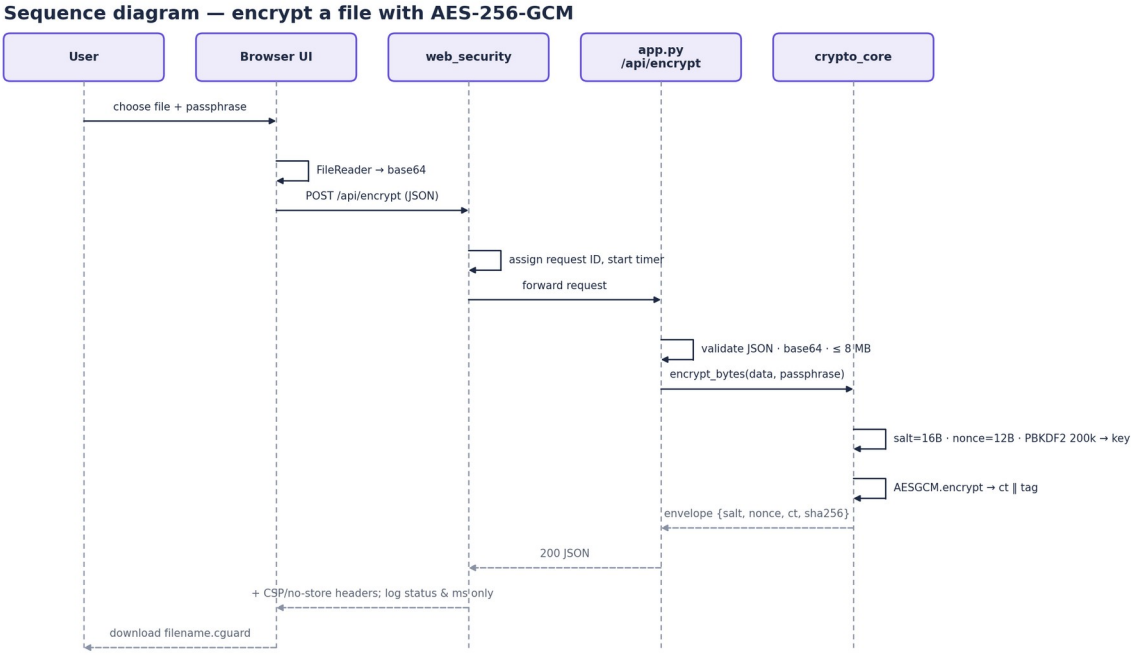


Figure 4.5: Sequence diagram for AES-256-GCM file encryption

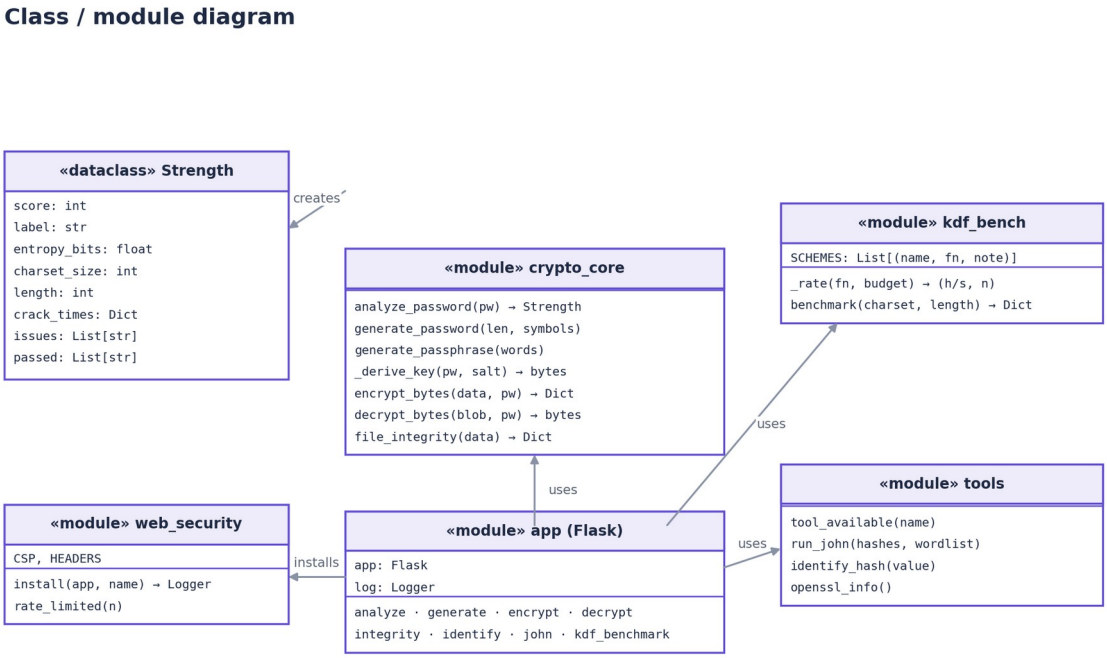


Figure 4.6: Class and module diagram

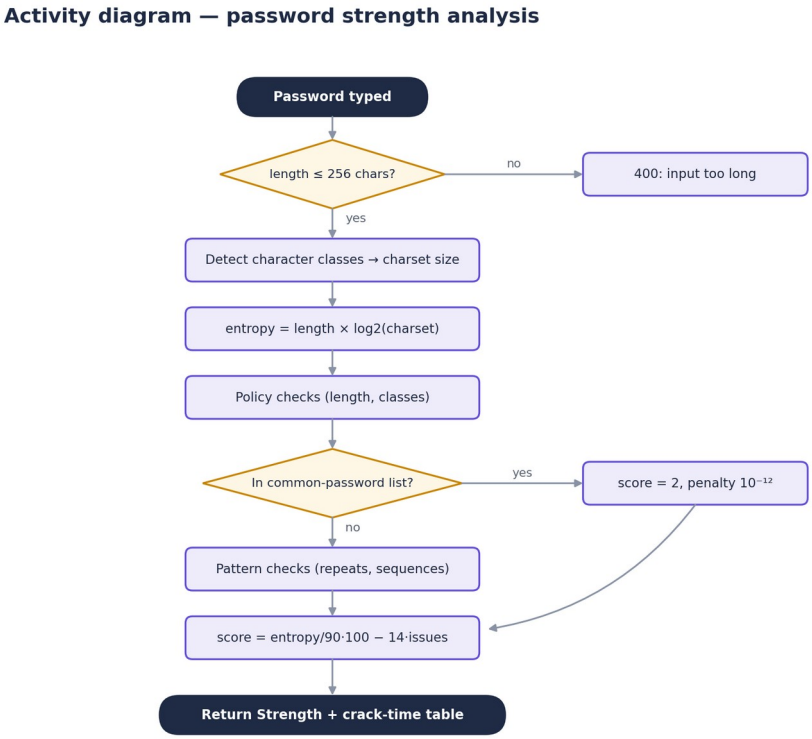


Figure 4.7: Activity diagram of password-strength analysis

4.4 Deployment Design

The application runs in one container built from `python:3.12-slim` (Figure 4.8). The container runs as an unprivileged user (uid 10001), publishes its port only on 127.0.0.1, has a health check, and mounts no volumes, so nothing a user uploads can persist.

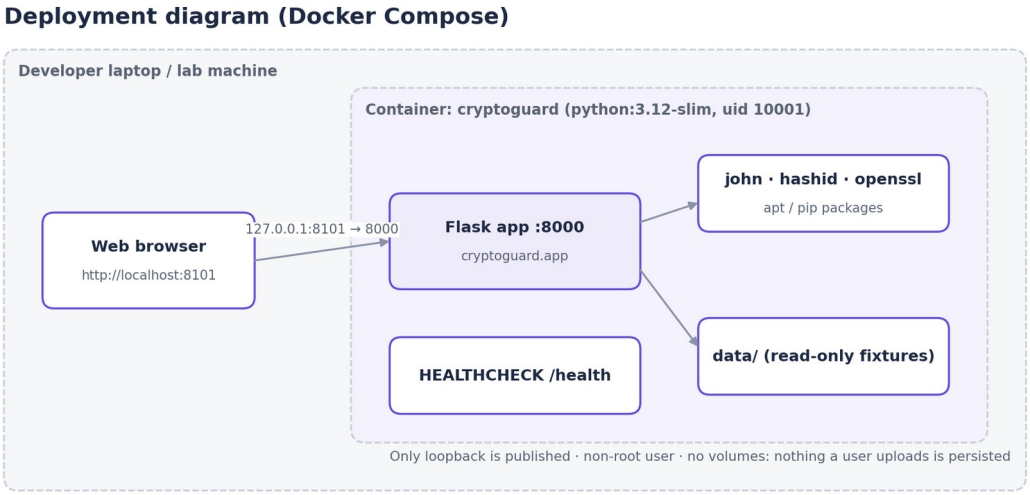


Figure 4.8: Deployment diagram

4.5 Encrypted Envelope Format

Table 4.1: Fields of the .cguard envelope

Field	Content	Secret?
ciphertext_b64	base64(salt[16] ‖ nonce[12] ‖ ciphertext ‖ GCM tag[16])	No (protected by key)
algorithm / kdf	"AES-256-GCM" / "PBKDF2-HMAC-SHA256 (200,000 iters)"	No
salt_hex, nonce_hex	Random per encryption (os.urandom)	No — safe to store
sha256	Digest of the plaintext, for integrity checks after decryption	No
filename, size	Original name (≤ 120 chars) and length	No
passphrase	**Never stored** in the envelope or on the server	Yes

CHAPTER 5

METHODOLOGY AND IMPLEMENTATION

5.1 Technology Stack and Justification

Table 5.1: Technology choices

Technology	Role	Why this and not something heavier
Python 3.12	Backend language	Readable; first-class security libraries
Flask 3	HTTP API	Minimal; every route visible in one file; no ORM needed (no database)
cryptography (OpenSSL)	AES-GCM, PBKDF2, scrypt	Vetted primitives; never hand-rolled crypto
secrets / hashlib	CSPRNG, digests	Standard library; no extra dependency
John the Ripper, hashid, OpenSSL CLI	Real security tooling	Industry tools, wrapped safely
Vanilla HTML/CSS/JS	User interface	No build step; one file; strict CSP possible
Docker + Compose	Packaging	Identical toolchain on every machine
pytest + GitHub Actions	Testing and CI	Fast tests on every push

Deliberately absent: a database (nothing must persist), a JavaScript framework (would complicate the CSP and add a build chain) and user accounts (single-user loopback tool). Each omission removes an attack surface.

5.2 Stage 1 — Project Skeleton and API

What: a Flask app that serves the UI and a `/health` endpoint, with a `_body()` helper that accepts only JSON objects. **Why:** every later feature needs a validated request path. **How:** all endpoints return JSON errors with HTTP 400 for malformed input instead of raising exceptions, and the request size is capped by `MAX_CONTENT_LENGTH`.

5.3 Stage 2 — Explainable Password Analysis

What: `analyze_password()` estimates entropy as length × log2(charset), applies policy checks, detects common passwords, repeats and keyboard sequences, and converts the keyspace into crack-time scenarios at four stated guess rates. **Why:** the user should see *why* a score was given and which assumptions sit behind each time. **How:** every rule contributes either to `issues` or `passed`, and the UI lists both.

Listing 5.2: Password strength analysis (cryptoguard/crypto_core.py)

```
def analyze_password(pw: str) -> Strength:
    length = len(pw)
    charset = 0
    if re.search(r"[a-z]", pw):
```

```
        charset += 26
    if re.search(r"[A-Z]", pw):
        charset += 26
    if re.search(r"\d", pw):
        charset += 10
    if re.search(r"[^A-Za-z0-9]", pw):
        charset += 33
    charset = max(charset, 1)
    entropy = length * math.log2(charset) if length else 0.0

    issues, passed = [], []
    (passed if length >= 12 else issues).append(
        "at least 12 characters" if length >= 12 else f"too short ({length} chars) — use
12+")
    for name, rx in [("lowercase", r"[a-z]"), ("uppercase", r"[A-Z]"),
                     ("a digit", r"\d"), ("a symbol", r"[^A-Za-z0-9]")]:
        (passed if re.search(rx, pw) else issues).append(
            f"contains {name}" if re.search(rx, pw) else f"add {name}")
    if pw.lower() in COMMON:
        issues.append("appears in common-password lists")
    else:
        passed.append("not a known common password")
    if re.search(r"(.)\1{2,}", pw):
        issues.append("repeated characters (e.g. aaaa)")
    if re.search(r"(?:0123|1234|2345|3456|4567|5678|6789|abcd|qwer)", pw.lower()):
        issues.append("predictable sequence (e.g. 1234, qwer)")

    # Effective guesses ~ half the keyspace; penalise detected weaknesses.
    penalty = 1.0
    if pw.lower() in COMMON:
        penalty = 1e-12
    guesses = (charset ** length) / 2 * penalty if length else 1

    crack_times = {k: _human_time(guesses / rate) for k, rate in _GUESS_RATES.items()}

    score = max(0, min(100, int(entropy / 90 * 100) - 14 * (len(issues))))
    if pw.lower() in COMMON:
        score = 2
    label = (["Very weak", "Weak", "Fair", "Strong", "Very strong"]
             [min(4, score // 20)])
    return Strength(score, label, round(entropy, 1), charset, length,
                    crack_times, issues, passed)
```

5.4 Stage 3 — Secure Secret Generation

Passwords and passphrases are drawn with `secrets.choice`, which uses the operating system's CSPRNG. The ordinary `random` module is predictable from its outputs and must never be used for secrets. Inputs are bounded (12–64 characters, 4–8 words) at the API.

5.5 Stage 4 — Authenticated Encryption

What: `encrypt_bytes()` derives a 256-bit key with PBKDF2-HMAC-SHA256 (200,000 iterations) from the passphrase and a fresh 16-byte salt, then encrypts with AES-GCM under a fresh 12-byte nonce. **Why:** GCM gives integrity as well as confidentiality, so a modified envelope or wrong passphrase is rejected rather than decrypted to garbage. **How:** salt and nonce are prepended to the ciphertext; both are public values. Decryption errors return a single generic message so that the API does not reveal *why* decryption failed.

Listing 5.3: Key derivation and AES-256-GCM (cryptoguard/crypto_core.py)

```
def _derive_key(password: str, salt: bytes) -> bytes:
    kdf = PBKDF2HMAC(algorithm=hashes.SHA256(), length=32, salt=salt, iterations=200_000)
    return kdf.derive(password.encode())
```

```
def encrypt_bytes(data: bytes, password: str) -> Dict:
    salt = os.urandom(16)
    nonce = os.urandom(12)
    key = _derive_key(password, salt)
    ct = AESGCM(key).encrypt(nonce, data, None)
    blob = salt + nonce + ct
    return {
        "ciphertext_b64": base64.b64encode(blob).decode(),
        "algorithm": "AES-256-GCM",
        "kdf": "PBKDF2-HMAC-SHA256 (200,000 iters)",
        "salt_hex": salt.hex(),
        "nonce_hex": nonce.hex(),
        "sha256": hashlib.sha256(data).hexdigest(),
        "size": len(data),
    }

def decrypt_bytes(blob_b64: str, password: str) -> bytes:
    blob = base64.b64decode(blob_b64)
    salt, nonce, ct = blob[:16], blob[16:28], blob[28:]
    key = _derive_key(password, salt)
    return AESGCM(key).decrypt(nonce, ct, None)
```

5.6 Stage 5 — Integrity and Hash Identification

File digests are computed with `hashlib`. Hash identification combines a small set of regular expressions (MD5/NTLM, SHA-1, SHA-256, SHA-512, bcrypt, md5crypt, sha512crypt, Argon2) with the `hashid` tool when installed, so the feature degrades gracefully outside Docker.

5.7 Stage 6 — Bounded John the Ripper Demonstration

What: a wrapper runs `john --wordlist` and `john --show` on the bundled md5crypt sample file. **Why:** seeing a weak hash fall in seconds teaches more than any meter. **How:** the endpoint accepts no parameters; file paths are constants; subprocesses use argument lists (no shell) and timeouts; the pot file is cleared so each demonstration is fresh.

Listing 5.4: John the Ripper adapter, beginning (cryptoguard/tools.py)

```
def run_john(hashes_path: str, wordlist_path: str, fmt: str = None) -> Dict:
    """Crack bundled md5crypt sample hashes with John (format auto-detected).

    Core John stores results in the default pot (~/.john/john.pot); we clear it
    first so the demo always shows a fresh crack.
    """
    if not tool_available("john"):
        return {"available": False, "note": "John the Ripper not installed in this
environment."}
    # Core John stores results in <home>/.john/john.pot and resolves <home> from
    # the password database (not always $HOME). Resolve it the same way, set HOME
    # to match, and clear the pot so every demo run is a fresh crack.
    try:
        import pwd as _pwd
        home = _pwd.getpwuid(os.getuid()).pw_dir
    except Exception:
        home = os.path.expanduser("~")
    env = {**os.environ, "HOME": home}
    pot = os.path.join(home, ".john", "john.pot")
    try:
        os.makedirs(os.path.dirname(pot), exist_ok=True)
        if os.path.exists(pot):
            os.remove(pot)
    except OSError:
        pass
```

```
    cmd = ["john", "--wordlist=%s" % wordlist_path]
    show = ["john", "--show"]
    if fmt:
        cmd.append("--format=%s" % fmt)
        show.append("--format=%s" % fmt)
    try:
        run = subprocess.run([*cmd, hashes_path], capture_output=True, text=True, timeout=60,
env=env)
        shown = subprocess.run([*show, hashes_path], capture_output=True, text=True,
timeout=30, env=env)
    except (subprocess.TimeoutExpired, OSError) as exc:
        return {"available": True, "error": str(exc)}
    # … (trimmed; see repository for the full function)
```

5.8 Stage 7 — Measured KDF Benchmark (Novel Component)

What: `kdf_bench.benchmark()` times six schemes on the running machine for about 0.15 s each and reports guesses per second. **Why:** it replaces an assumed attacker speed with a measured one, and isolates the effect of the storage scheme. **How:** for a password model with alphabet size *c* and length *n*, the keyspace is c^n. If scheme *S* runs at r_S guesses/s and MD5 at r_MD5, the slowdown is k = r_MD5 / r_S. Because each extra random character multiplies the keyspace by *c*, the slowdown is equivalent to **log(k) / log(c)** extra characters. The password itself is used only to choose *c* and *n*; it is never hashed or logged.

Listing 5.5: Benchmark core (cryptoguard/kdf_bench.py)

```
def _rate(fn: Callable[[bytes], bytes], budget_s: float) -> Tuple[float, int]:
    """Return (hashes per second, samples) by timing fn for about budget_s."""
    count = 0
    start = time.perf_counter()
    elapsed = 0.0
    while elapsed < budget_s or count < 2:
        fn(b"candidate-%d" % count)
        count += 1
        elapsed = time.perf_counter() - start
    return count / elapsed, count

def benchmark(charset_size: int = 95, length: int = 10, budget_s: float = 0.15) -> Dict:
    if not 1 <= charset_size <= 256 or not 1 <= length <= 128:
        raise ValueError("charset_size must be 1-256 and length 1-128")
    keyspace_log10 = length * math.log10(charset_size)
    rows = []
    for name, fn, note in SCHEMES:
        rate, samples = _rate(fn, budget_s)
        rows.append({"scheme": name, "note": note,
                     "hashes_per_second": round(rate, 1), "samples": samples})
    base = rows[0]["hashes_per_second"]
    for row in rows:
        slowdown = base / row["hashes_per_second"]
        seconds_log10 = keyspace_log10 - math.log10(row["hashes_per_second"])
        row["slowdown_vs_md5"] = round(slowdown, 1)
        row["extra_chars_equivalent"] = round(math.log(slowdown) / math.log(charset_size), 2)
if charset_size > 1 else 0.0
        row["exhaust_keyspace"] = _human(10 ** seconds_log10) if seconds_log10 < 300 else
f"10^{seconds_log10:.0f} seconds"
    return {
        "machine": f"{platform.system()} {platform.machine()}, Python
{platform.python_version()}, one core",
        "password_model": {"charset_size": charset_size, "length": length,
                           "keyspace": f"{charset_size}^{length} ≈ 10^{keyspace_log10:.1f}"},
        "results": rows,
        "caveat": "Measured on this machine with hashlib; GPU/ASIC attackers are much faster
on fast hashes. Compare ratios, not absolute times.",
    }
```

5.9 Stage 8 — HTTP Hardening, Logging and Rate Limiting

`web_security.install()` adds OWASP-recommended headers [14] to every response: a Content-Security-Policy restricted to the app's own origin, `frame-ancestors 'none'` and `X-Frame-Options: DENY` against clickjacking, `nosniff`, `no-referrer` and a restrictive Permissions-Policy. API responses are marked `Cache-Control: no-store`. Each request receives an ID (a client-supplied ID is accepted only if it matches `[A-Za-z0-9-]{1,64}`, preventing log injection), and one log line records method, path, status and duration, never the body. Unhandled exceptions are logged server-side and return a generic 500 with the request ID. The `rate_limited(n)` decorator implements a per-client sliding window and returns HTTP 429 with `Retry-After`.

Listing 5.6: Rate-limiting decorator (cryptoguard/web_security.py)

```
def rate_limited(per_minute: int):
    """Allow at most ``per_minute`` calls per client address for this endpoint."""
    def decorator(view):
        @wraps(view)
        def wrapper(*args, **kwargs):
            key = (view.__name__, request.remote_addr or "?")
            now = time.monotonic()
            with _lock:
                window = _hits[key]
                while window and now - window[0] > 60:
                    window.popleft()
                if len(window) >= per_minute:
                    retry = int(60 - (now - window[0])) + 1
                    response = jsonify({"error": "rate limit reached; try again shortly",
                                        "retry_after_s": retry})
                    response.status_code = 429
                    response.headers["Retry-After"] = str(retry)
                    return response
                window.append(now)
            return view(*args, **kwargs)
        return wrapper
    return decorator
```

5.10 Stage 9 — User Interface

The interface is one HTML file with tabbed tools, live analysis as the user types (debounced at 180 ms), drag-and-drop file handling in browser memory, and a results table for the benchmark. All server-provided strings are HTML-escaped before insertion. At widths below 720 px the benchmark table collapses to one card per scheme, and buttons grow to a 44 px minimum touch target.

5.11 Stage 10 — Packaging and Continuous Integration

The Dockerfile installs John the Ripper and OpenSSL, creates an unprivileged user, and defines a health check; Compose binds the port to 127.0.0.1:8101. A GitHub Actions workflow installs dependencies and runs the test suite on every push and pull request.

5.12 Secure Coding Practices Applied

- Validate type, length and range of every input; reject non-object JSON bodies.
- Use vetted primitives (AES-GCM, PBKDF2, scrypt from OpenSSL) and a CSPRNG; no custom cryptography.
- Fail closed on authentication-tag mismatch; return non-specific error messages.
- Call external tools with argument lists, constant paths and timeouts; never through a shell.
- Never log request bodies; sanitise request IDs before logging.
- Apply defence-in-depth headers and per-client rate limits on CPU-expensive endpoints.
- Run the container as non-root with only the loopback port published.

CHAPTER 6

TESTING AND VALIDATION

6.1 Test Strategy

Table 6.1: Test levels used

Level	Purpose	Tooling
Unit	Pure functions in crypto_core, tools and kdf_bench	pytest
Integration	HTTP routes end-to-end through Flask's test client	pytest + Flask test client
Security	Headers, input bounds, log-injection guard, rate limits, error format	pytest (tests/test_security.py)
Functional / UI	Real browser flows on the Docker build, including real tools	Playwright-driven Chromium; screenshots
Performance	Throughput and latency under concurrent load	scripts/loadtest.py (thread pool, stdlib only)

6.2 Automated Test Cases

Table 6.2 lists the automated test cases with the pytest function that implements each. The suite was executed on 14 tests with the result **14 passed, 0 failed**.

Table 6.2: Automated test cases and results

ID	Type	Scenario	Expected result	Actual (test function)
TC-01	Unit	Weak vs strong password scoring	"password" < 10; random 16-char > 80 and > 80 bits	**PASS** (test_weak_vs_strong)
TC-02	Unit	AES-GCM round trip and wrong key	Plaintext restored; wrong key raises	**PASS** (test_encrypt_roundtrip_and_tamper)
TC-03	Unit	Generator length and digest size	20 chars; passphrase has '-'; SHA-256 = 64 hex	**PASS** (test_generator_and_integrity)
TC-04	Unit	Built-in hash identification	MD5 sample recognised	**PASS** (test_hash_identify_builtin)
TC-05	Integration	API health, analyse, encrypt → decrypt	200 responses; decrypted text matches	**PASS** (test_api_endpoints)
TC-06	Security	Oversized and malformed inputs	HTTP 400 for 257-char password, bad base64, out-of-range generator	**PASS** (test_api_rejects_bad_input_and_bounds_generator)
TC-07	Integration	File envelope keeps filename and decrypts	Algorithm AES-256-GCM; bytes restored	**PASS** (test_cguard_envelope_includes_filename_and_decrypts)
TC-08	Unit	KDF benchmark ordering	MD5 > 1000× PBKDF2-200k; 200k > 600k; 600k worth > 2 chars	**PASS** (test_kdf_benchmark_orders_schemes_by_cost)

ID	Type	Scenario	Expected result	Actual (test function)
TC-09	Security	KDF endpoint validation	HTTP 400 for 300-char password and non-JSON body	**PASS** (test_kdf_endpoint_validates_input)
TC-10	Security	Security headers on page and API	CSP default-src 'self', frame-ancestors 'none', DENY, nosniff, no-referrer	**PASS** (test_security_headers_on_page_and_api)
TC-11	Security	API responses not cached	Cache-Control: no-store	**PASS** (test_api_responses_are_not_cached)
TC-12	Security	Request-ID log-injection guard	Safe ID echoed; forged ID replaced by 12-char token	**PASS** (test_request_id_is_echoed_only_when_safe)
TC-13	Security	Unknown API route	JSON 404 body	**PASS** (test_unknown_api_route_returns_json_error)
TC-14	Security	Rate limit on John endpoint	7th call within a minute returns 429	**PASS** (test_expensive_endpoint_is_rate_limited)

6.3 Functional, Usability and Performance Checks

Table 6.3: Functional and non-functional checks

ID	Type	Scenario	Expected result	Actual
TC-15	Functional	Live strength meter on typing	Score ring, checks and bars update	**PASS** (Fig. 7.2)
TC-16	Functional	Encrypt text in UI and download envelope	Envelope JSON shown; download offered	**PASS** (Fig. 7.3)
TC-17	Functional	John demo in Docker	Cracked list shown (8/9)	**PASS** (Fig. 7.4)
TC-18	Functional	KDF benchmark in UI	Table with 6 schemes and char equivalents	**PASS** (Fig. 7.5)
TC-19	Usability	Layout at 390 px width	No horizontal scroll; controls ≥ 44 px	**PASS** (Fig. 7.6)
TC-20	Performance	Load test, 100 requests × 4 endpoints	0 errors; p95 < 50 ms except KDF-bound encrypt	**PASS** (Table 7.3)

6.4 Security Testing Notes

Beyond the automated checks, the following were verified by inspection: the John endpoint takes no request data (so it cannot be pointed at other files); decryption errors return the same message for wrong passphrase and tampering; the access log contains no request bodies; the container user is uid 10001; and the only published port is on 127.0.0.1. The Content-Security-Policy was observed in practice: the browser automation tool used for screenshots

was itself blocked from evaluating injected script until CSP bypass was enabled for that tool only.

CHAPTER 7

RESULTS AND DISCUSSION

7.1 Working System

Figures 7.1 to 7.6 show the system running from the Docker image with all external tools installed.

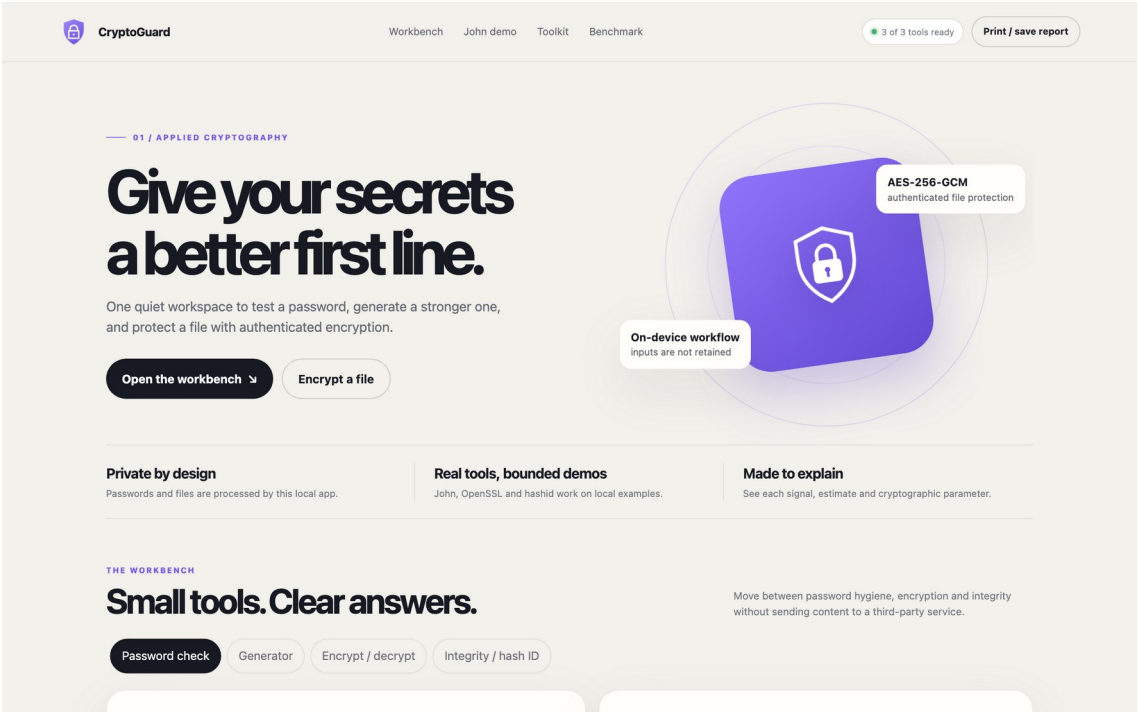


Figure 7.1: CryptoGuard landing view

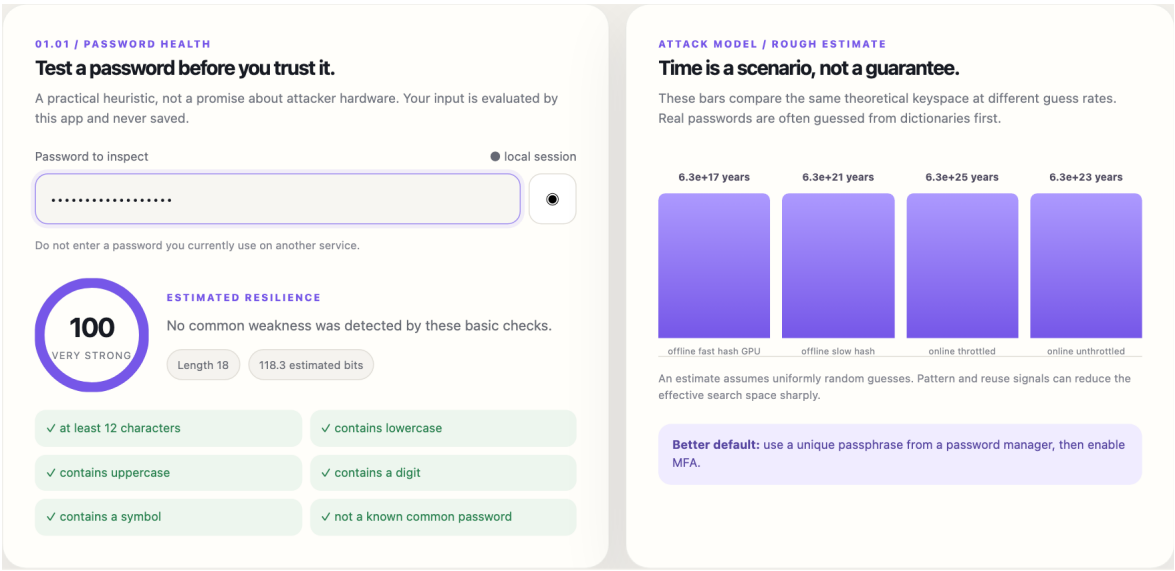


Figure 7.2: Live password analysis with checks and crack-time scenarios

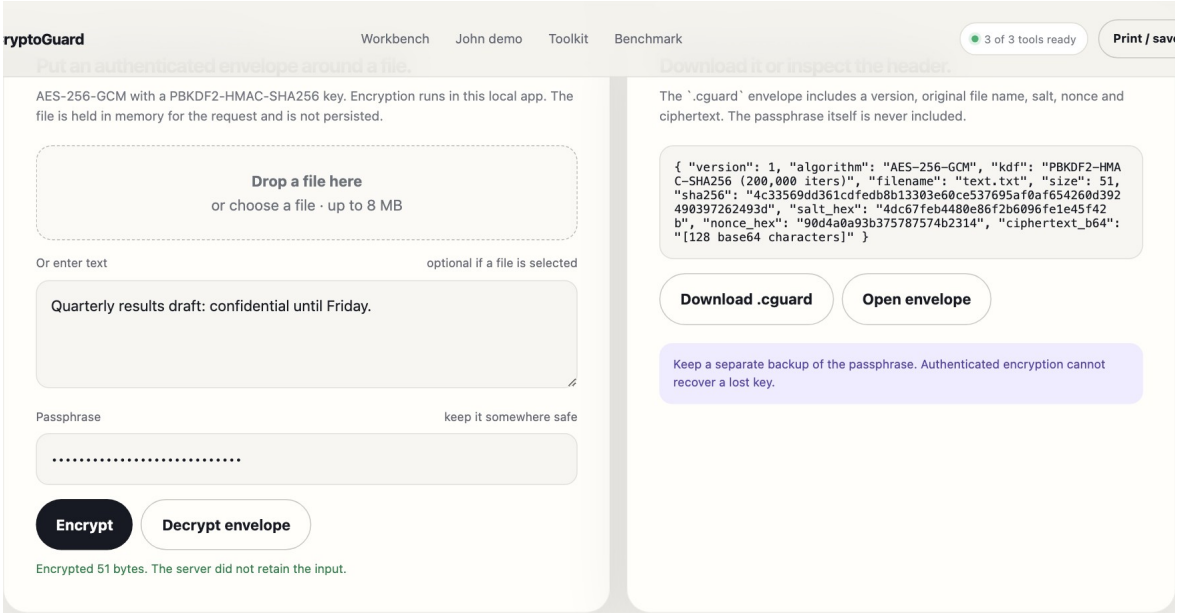


Figure 7.3: AES-256-GCM encryption producing a .cguard envelope

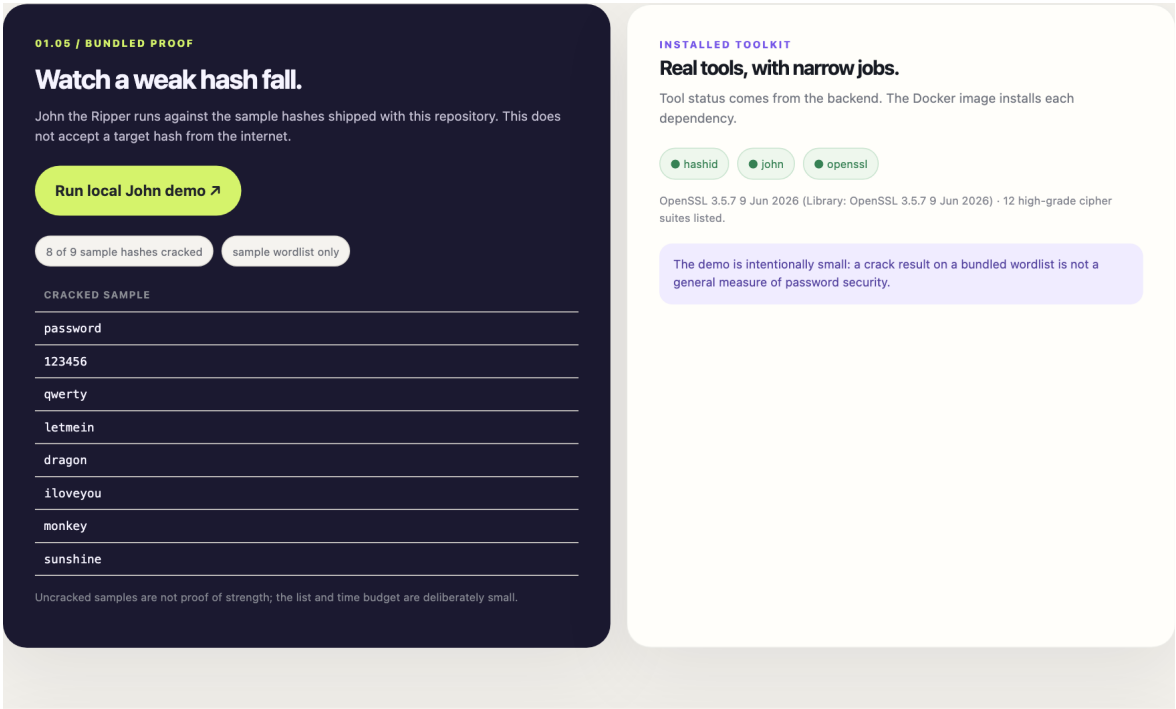


Figure 7.4: John the Ripper cracking 8 of 9 bundled sample hashes

7.2 Measured KDF Cost

Table 7.1 and Figure 7.7 give the benchmark measured inside the Docker image (Linux aarch64, Python 3.12.16, one core). The three fast hashes run at roughly 3.5–4.2 million guesses per second per core, and their relative order varies by a few tens of percent between runs (Python call overhead dominates at this speed). The KDFs are slower by four to five orders of magnitude.

Table 7.1: Measured guess rates (Linux aarch64, Python 3.12.16, one core)

Scheme	Guesses/s	Slowdown vs MD5	Worth (chars, c = 95)	Note
MD5 (unsalted)	3,516,182	≈ 1	—	legacy fast hash; never use for passwords
SHA-1 (unsalted)	4,160,727	≈ 1	—	legacy fast hash
SHA-256 + salt	3,834,116	≈ 1	—	salted but still fast
PBKDF2-SHA256 200k	45.7	76,940×	+2.47	what CryptoGuard uses for file keys
PBKDF2-SHA256 600k	15.2	231,328×	+2.71	OWASP 2023 minimum for PBKDF2-SHA256
scrypt N=2^14 r=8	51.7	68,011×	+2.44	memory-hard: 16 MiB per guess

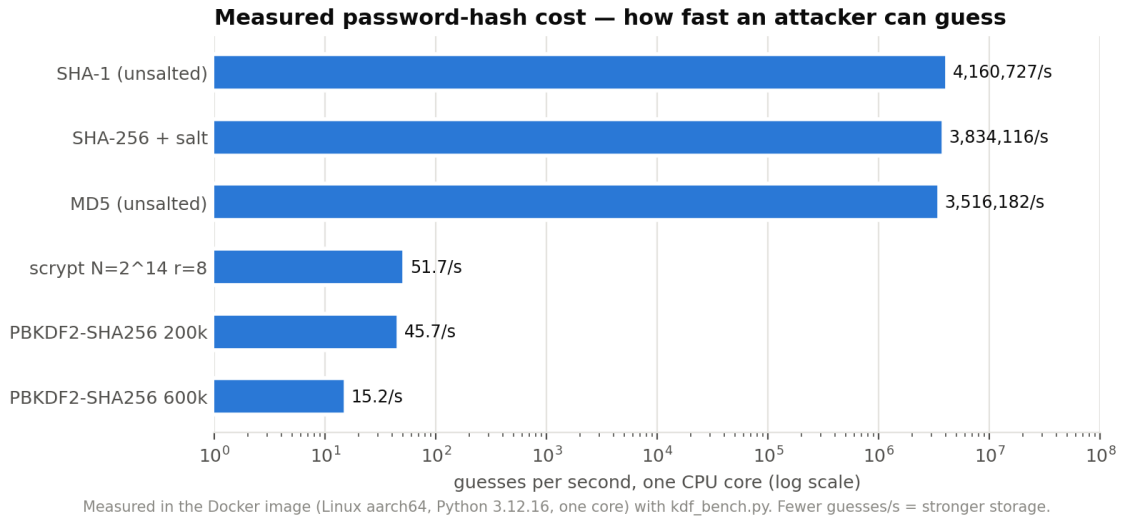


Figure 7.5: Guesses per second per scheme (log scale)

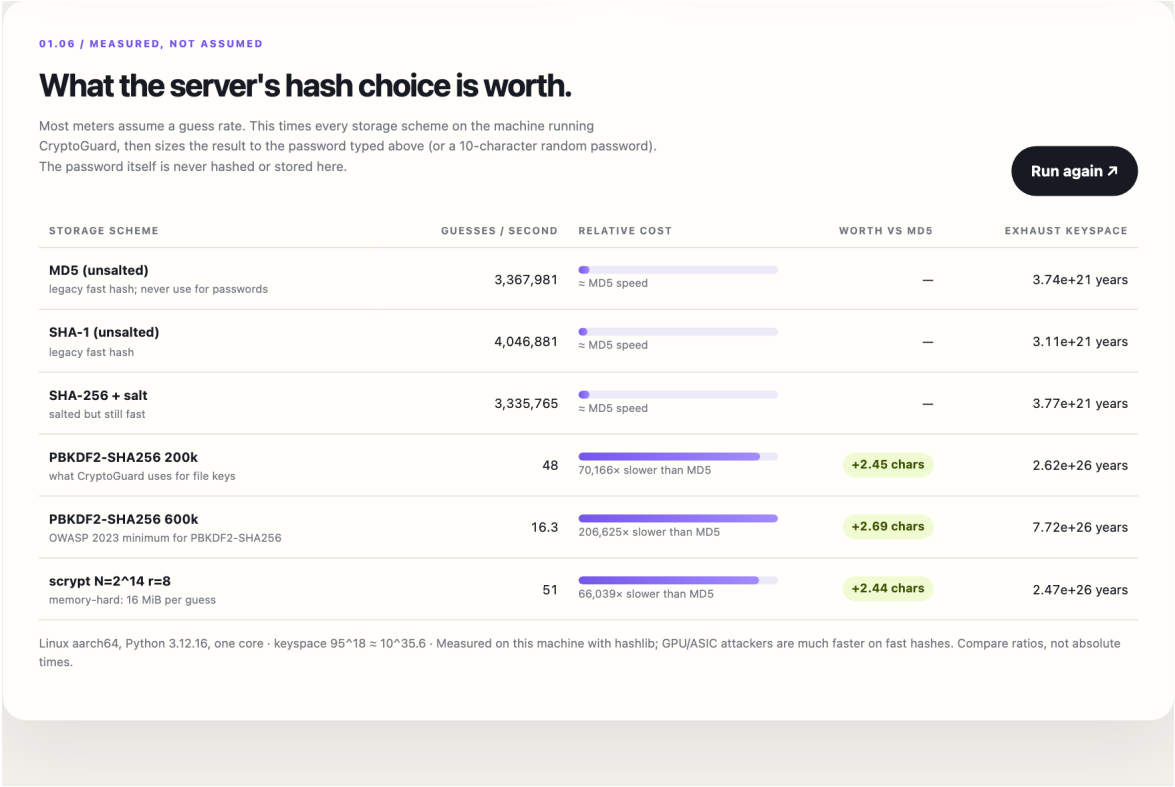


Figure 7.6: Benchmark rendered in the user interface

Interpretation. Moving from unsalted MD5 to PBKDF2-SHA256 with 600,000 iterations slows a single-core attacker by about 231,328×. For a password drawn uniformly from 95 printable characters, this equals **2.71 additional characters**: the server's choice is worth roughly as much as the user typing three more random symbols, at no cost to the user. scrypt at $N = 2^{14}$ is slightly *faster* per guess than PBKDF2-200k on one CPU core (51.7 vs 45.7 guesses/s), but its 16 MiB memory requirement is what limits GPU and ASIC parallelism [5]; a single-core CPU benchmark does not capture that advantage, which is an important limitation of this measurement.

The same benchmark on the development laptop's system Python (macOS, LibreSSL) gave lower absolute rates but the same ordering and similar character equivalents (+2.5 to +2.8), supporting the claim that the *ratios* are the robust result.

7.3 Strength Analyser Behaviour

The analyser scored the common password "password" at 2/100 (blocklisted), and "Tr0ub4dor&3-Horse!" at 100/100 with 118.3 estimated bits. The second result exposes a known weakness of charset-entropy estimators, discussed by Weir et al. [8] and addressed by zxcvbn [7]: a dictionary word with predictable l33t substitutions is far weaker than its

character count suggests. This is reported as a limitation (Section 7.6) rather than hidden, and it is why the UI labels crack times as rough estimates.

7.4 Performance

Table 7.2: Load-test results against the Docker container (100 requests, 8 workers per endpoint)

Endpoint	Req/s	p50 ms	p95 ms	max ms	Errors
password analysis	1,535	4.8	7.8	8.4	0
secret generation	1,638	4.5	7.0	9.3	0
file integrity (1 KB)	1,761	4.3	6.6	7.3	0
AES-GCM encrypt (1 KB, PBKDF2 200k)	318	23.7	31.0	36.4	0

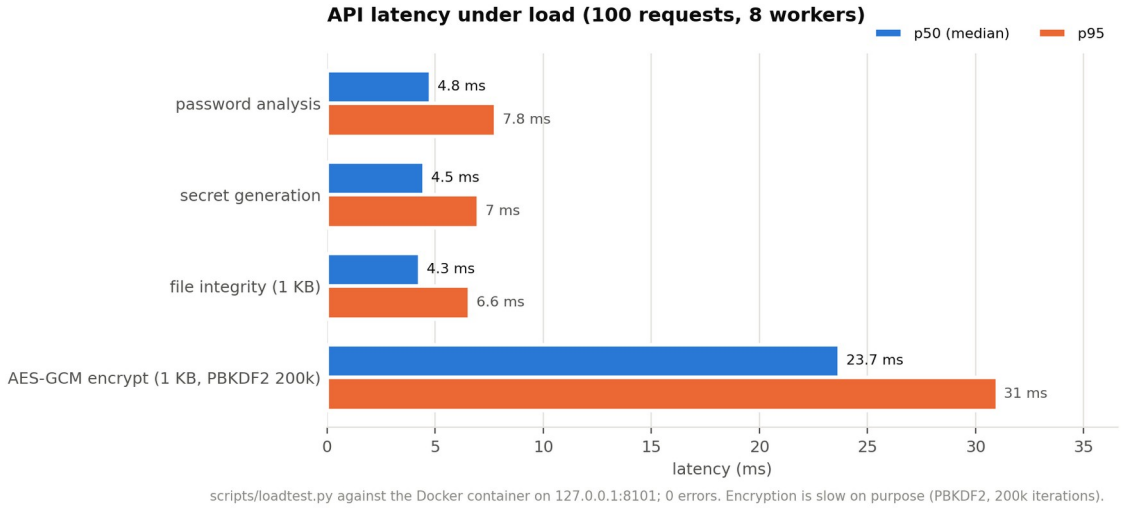


Figure 7.7: Latency per endpoint (p50 and p95)

All requests succeeded. The encryption endpoint is about five times slower than the others because PBKDF2 with 200,000 iterations runs on every request: this is the intended cost, and it is the same cost an attacker pays per guess. NFR-4 is met.

7.5 Usability on Mobile

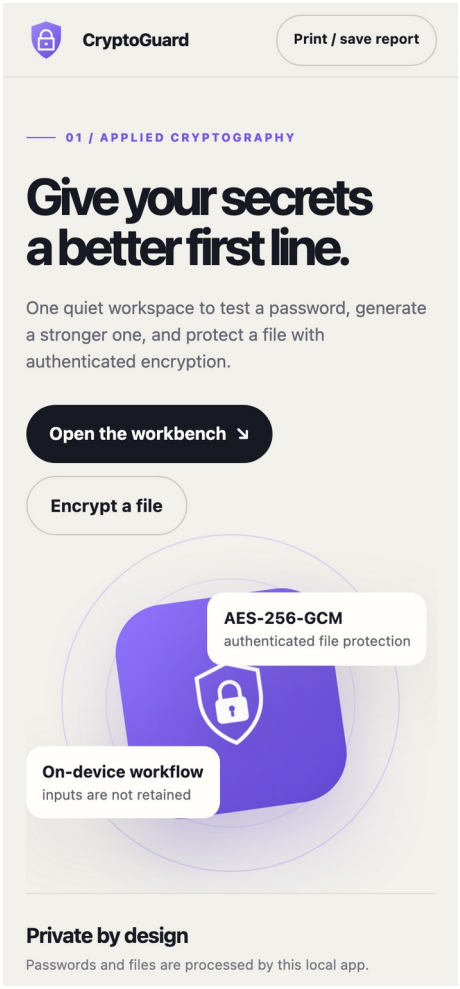


Figure 7.8: Layout at 390 px width

7.6 Limitations

- The strength score is a teaching heuristic, not a guess-number estimator; it over-rates dictionary words with l33t substitutions.
- The benchmark measures one CPU core in Python. It understates attacker speed on GPUs for fast hashes and does not capture scrypt's memory-hardness advantage.
- Argon2 and bcrypt are not benchmarked because they require additional libraries.
- The envelope format is versioned only implicitly; it has not been reviewed for interoperability.
- The application has no authentication and must stay on loopback; it has not had an independent security audit.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

CryptoGuard meets its objectives: it analyses, generates, encrypts, verifies and cracks within a single local, hardened application, and it adds a measured view of password storage that typical meters lack. The central finding is quantitative and reproducible: on the test machine, choosing PBKDF2-SHA256 with 600,000 iterations instead of MD5 slowed guessing by about 231,328×, equivalent to roughly 2.7 extra random password characters. All 14 automated tests pass, and the system performs well within its requirements.

As the first design project, it establishes the engineering foundations used in the later projects (PhishLens and ThreatScope): input validation, safe tool integration, honest evaluation, containerisation and continuous testing.

8.2 Future Scope

- Replace the heuristic with a guess-number estimator (zxcvbn-style pattern matching) and compare both against a public cracking benchmark.
- Add bcrypt and Argon2id to the benchmark and a GPU measurement with Hashcat to model attacker hardware.
- Check passwords against breach corpora with k-anonymity range queries, opt-in only.
- Define a versioned, documented envelope format with interoperability tests.
- Add an optional authenticated multi-user mode for classroom deployment.

REFERENCES

[1] P. A. Grassi, J. L. Fenton, E. M. Newton, R. A. Perlner, A. R. Regenscheid, W. E. Burr, J. P. Richer, N. B. Lefkovitz, J. M. Danker, Y.-Y. Choong, K. K. Greene and M. F. Theofanos, “Digital Identity Guidelines: Authentication and Lifecycle Management,” NIST Special Publication 800-63B, National Institute of Standards and Technology, 2017.

[2] M. S. Turan, E. Barker, W. Burr and L. Chen, “Recommendation for Password-Based Key Derivation, Part 1: Storage Applications,” NIST Special Publication 800-132, 2010.

[3] M. Dworkin, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” NIST Special Publication 800-38D, 2007.

[4] K. Moriarty, B. Kaliski and A. Rusch, “PKCS #5: Password-Based Cryptography Specification Version 2.1,” IETF RFC 8018, 2017.

[5] C. Percival and S. Josefsson, “The scrypt Password-Based Key Derivation Function,” IETF RFC 7914, 2016.

[6] J. Bonneau, “The science of guessing: analyzing an anonymized corpus of 70 million passwords,” in Proc. IEEE Symposium on Security and Privacy, 2012, pp. 538–552.

[7] D. L. Wheeler, “zxcvbn: Low-budget password strength estimation,” in Proc. 25th USENIX Security Symposium, 2016, pp. 157–173.

[8] M. Weir, S. Aggarwal, M. Collins and H. Stern, “Testing metrics for password creation policies by attacking large sets of revealed passwords,” in Proc. 17th ACM Conference on Computer and Communications Security (CCS), 2010, pp. 162–175.

[9] OWASP Foundation, “Password Storage Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[10] Openwall, “John the Ripper password cracker.” [Online]. Available: https://www.openwall.com/john/

[11] A. Biryukov, D. Dinu and D. Khovratovich, “Argon2: new generation of memory-hard functions for password hashing and other applications,” in Proc. IEEE European Symposium on Security and Privacy (EuroS&P), 2016, pp. 292–302.

[12] B. Ur, J. Bees, S. M. Segreti, L. Bauer, N. Christin and L. F. Cranor, “Do users' perceptions of password security match reality?,” in Proc. ACM CHI Conference on Human Factors in Computing Systems, 2016, pp. 3748–3760.

[13] Python Cryptographic Authority, “cryptography” library documentation. [Online]. Available: https://cryptography.io/

[14] OWASP Foundation, “OWASP Secure Headers Project.” [Online]. Available: https://owasp.org/www-project-secure-headers/

[15] P. G. Kelley, S. Komanduri, M. L. Mazurek, R. Shay, T. Vidas, L. Bauer, N. Christin, L. F. Cranor and J. Lopez, “Guess again (and again and again): measuring password strength by simulating password-cracking algorithms,” in Proc. IEEE Symposium on Security and Privacy, 2012, pp. 523–537.

APPENDIX A: INSTALLATION AND USER MANUAL

9. Install Docker Desktop and start it.
10. Open a terminal in the project folder.
11. Run `docker compose up --build -d` and open http://localhost:8101.
12. Stop with `docker compose down`. On macOS you can instead double-click Start.command.

Listing A.1: Local development without Docker

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
cp .env.example .env      # optional
make test                 # run the test suite
make run                  # http://127.0.0.1:8000
python -m cryptoguard.kdf_bench   # benchmark from the command line
python scripts/loadtest.py        # needs the app running
```

Table A.2: Troubleshooting

Symptom	Cause and fix
John panel says not installed	Running outside Docker; use the container or install john
HTTP 429 on benchmark	Rate limit (6/min) reached; wait a minute
Port 8101 already in use	Stop the other process or change the port in docker-compose.yml
Decryption failed	Wrong passphrase or edited envelope; GCM rejects both by design

APPENDIX B: DEMONSTRATION SCRIPT

13. Open the app and point out the tool-status pill (all tools ready in Docker) and the privacy promise.
14. Type "password" — score near zero, blocklist issue shown. Then type a long random password — score rises; read out the four crack-time scenarios and their stated guess rates.
15. Generate a 20-character password and a passphrase; explain CSPRNG vs random.
16. Encrypt a short note with a passphrase; show salt, nonce and algorithm in the envelope; decrypt it; then change one character of the ciphertext and show decryption is refused.
17. Run the John demo: weak sample hashes fall in seconds from a 16-word list.
18. Run the KDF benchmark: MD5 vs PBKDF2-600k; read out the slowdown and the “+2.7 characters” result. Close with the key message: storage choices matter as much as password rules.

APPENDIX C: API REFERENCE

Table C.1: REST endpoints

Method and path	Input	Output
POST /api/analyze	{password}	score, label, entropy_bits, crack_times, issues, passed
GET /api/generate	type, length, symbols, words	{value}
POST /api/encrypt	{password, text | data_b64, filename}	envelope
POST /api/decrypt	{password, ciphertext_b64}	text, data_b64, sha256
POST /api/integrity	{text | data_b64}	sha256, sha1, md5, size
POST /api/identify	{hash}	builtin_guesses, hashid
GET /api/john	—	cracked, cracked_count, total
POST /api/kdf-bench	{password?}	machine, password_model, results[]
GET /api/tools, /api/openssl	—	tool availability; OpenSSL version and ciphers
GET /health	—	ok

APPENDIX D: REPOSITORY STRUCTURE

Listing D.1: Repository layout

```
├── .dockerignore
├── .env.example
├── .github/
│   └── workflows/
│       └── ci.yml
├── .gitignore
├── CHANGELOG.md
├── CONTRIBUTING.md
├── Dockerfile
├── LICENSE
├── Makefile
├── PROJECT_OVERVIEW.md
├── README.md
├── SECURITY.md
├── Start.command
├── cryptoguard/
│   ├── __init__.py
│   ├── app.py
│   ├── crypto_core.py
│   ├── data/
│   │   ├── sample_hashes.txt
│   │   └── wordlist.txt
│   ├── kdf_bench.py
│   ├── static/
│   │   └── index.html
│   ├── tools.py
│   └── web_security.py
├── docker-compose.yml
├── docs/
│   ├── README.md
│   ├── diagrams/  (10 images)
│   ├── index.html
│   └── screenshots/  (6 images)
├── requirements.txt
├── scripts/
│   └── loadtest.py
└── tests/
    ├── test_cryptoguard.py
    └── test_security.py
```